MERGE + Causal Clustering + Raft

Construcción de grafos, consenso distribuido y consistencia causal

Objetivos

Comprender MERGE, explicar cómo se confirma una transacción y analizar fallas en un clúster.

CREATE, MATCH y MERGE

CREATE

Siempre crea un patrón nuevo.

MATCH

Busca patrones existentes.

MERGE

Busca el patrón completo; si no existe, lo crea.

```
CREATE (p:Persona {nombre:'Ana'})
MATCH (p:Persona {nombre:'Ana'})
MERGE (p:Persona {nombre:'Ana'})
```

MERGE: comportamiento y propiedades

```
MERGE (p:Persona {id: 101})
ON CREATE SET p.nombre = 'Ana', p.alta = date()
ON MATCH SET p.ultimoAcceso = datetime()
RETURN p
```

- ON CREATE SET se ejecuta únicamente si se crea el patrón.
- ON MATCH SET se ejecuta cuando el patrón ya existe.
- SET fuera de esas cláusulas se ejecuta en ambos casos.

MERGE de relaciones: práctica recomendada




```
MERGE (a:Persona {id: 1})
MERGE (b:Persona {id: 2})
MERGE (a)-[:CONOCE]->(b)
```

Primero resolver los nodos; después la relación. MERGE opera sobre el patrón completo.

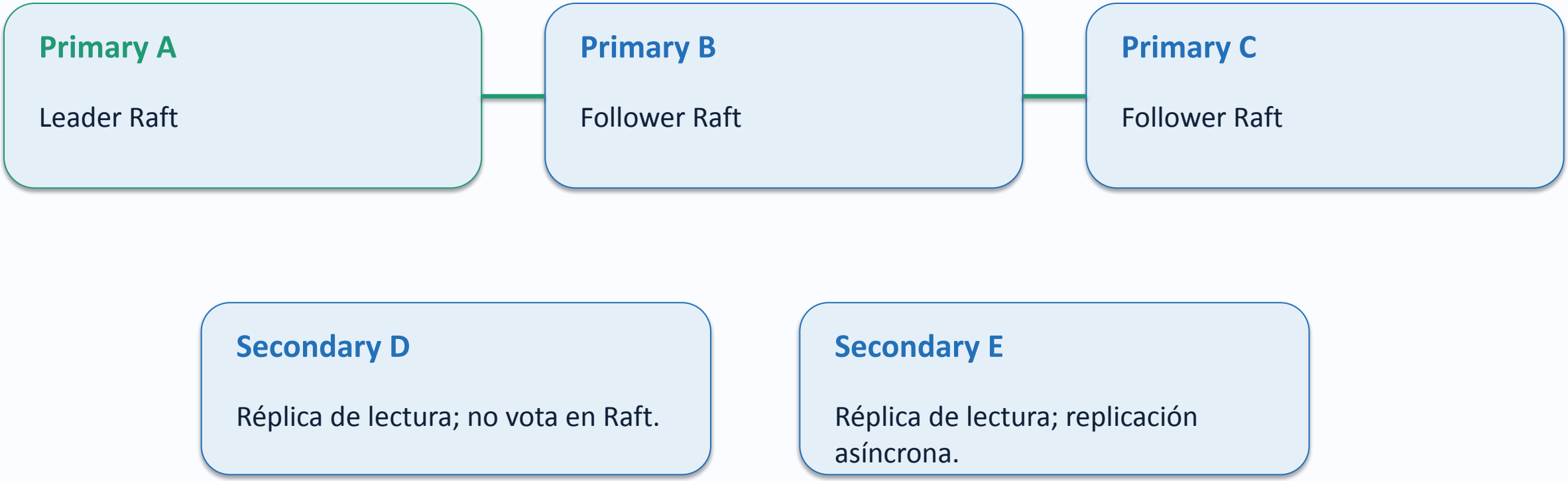
MERGE no reemplaza las restricciones

```
CREATE CONSTRAINT persona_id_unico IF NOT EXISTS
FOR (p:Persona) REQUIRE p.id IS UNIQUE;

MERGE (p:Persona {id: 101})
SET p.nombre = 'Ana';
```

- MERGE no elimina duplicados preexistentes.
- Concurrencia: usar una clave única y su restricción.
- Conviene evitar usar propiedades cambiantes como identidad.

Causal Clustering: arquitectura



Primarios: consenso y escrituras · Secundarios: escalado de lecturas

Raft: roles y elección de líder

Leader

Coordina el log de consenso de una base de datos.

Follower

Replica entradas y participa en votaciones.

Candidate

Solicita votos al comenzar una elección.

- Si falla el líder, los miembros restantes pueden elegir otro.
- Se requiere mayoría de votantes para elegir y confirmar.
- Raft coordina el orden de transacciones, no las lecturas secundarias.

Raft: ¿cómo se confirma una escritura?

1

El cliente envía la escritura

2

El líder agrega una entrada al log

3

Replica la entrada a los followers

4

La mayoría confirma recepción

5

La entrada se compromete y se aplica

Quórum y tolerancia a fallas

3 primarios

Quórum: 2
Fallas toleradas: 1

5 primarios

Quórum: 3
Fallas toleradas: 2

7 primarios

Quórum: 4
Fallas toleradas: 3

Regla para tolerar F fallas: $N \geq 2F + 1$

Sin mayoría, no pueden confirmarse nuevas escrituras mediante Raft.

¿Por qué causal? Bookmarks

Transacción T1

CREATE (:Persona {id:101})

Bookmark

El cliente conserva el punto de progreso de T1.

Transacción T2

La lectura espera una réplica suficientemente actualizada.

- Evita que una lectura causalmente dependiente observe un estado anterior.
- Los secundarios pueden presentar retraso de replicación.
- Se consigue la propiedad *Read Your Own Writes*.

Comparación: Neo4j y Cassandra

Neo4j (clúster)

Primarios con Raft; consenso con mayoría; secundarios para lecturas; bookmarks para causalidad.

Cassandra (operación habitual)

Arquitectura sin líder único de escritura; niveles de consistencia configurables; replicación entre nodos.

Cassandra usa mecanismos adicionales de consenso en operaciones específicas (por ejemplo, LWT).

Actividad para discutir en clase

- Tenemos 3 primarios y 2 secundarios. Falla el líder: ¿qué sucede?
- Luego falla otro primario: ¿pueden confirmarse escrituras?
- Un cliente escribe y lee desde un secundario: ¿qué riesgo existe?
- ¿Cómo ayuda un bookmark? ¿Y una restricción UNIQUE con MERGE?

Guía de respuestas y cierre

- Con 2 de 3 primarios activos, puede elegirse nuevo líder y mantener quórum.
- Con solo 1 de 3 primarios, no hay mayoría para nuevas escrituras.
- Un secundario puede ir atrasado; bookmarks permiten esperar el progreso causal.
- MERGE busca o crea patrones; UNIQUE protege la identidad ante concurrencia.
- Raft = consenso; Causal Clustering = arquitectura distribuida.